



Faculty of Electrical and Computer Engineering

**Design and Implementation of a Smart Home IoT System
with On-Device Edge AI Face Authentication
and a Fully Local, Cloud-Free Infrastructure**

Undergraduate Specialized Project
Computer Engineering

Supervisor:

Dr. Amir Khorsandi

Student:

Mohammad Soroush Rabiei

September 2026

Acknowledgments

I thank God, the Most Gracious, for granting me the opportunity to learn and to build.

I am sincerely grateful to my beloved parents, who have always supported me with kindness along the path of learning and effort.

I would also like to express my gratitude to my respected supervisor, Dr. Amir Khorsandi, whose invaluable guidance made this project and this report possible.

Abstract

In this project, a complete Internet of Things (IoT) smart-home system was designed and implemented with a distinctive focus on bringing artificial intelligence to low-power edge devices and reusing the hardware the user already owns. Face recognition — the authentication mechanism of the system — runs entirely on an ESP32-S3 microcontroller: the camera of the user's smartphone, reached by scanning a QR code shown on the device's touch LCD, acts as the image sensor, so no dedicated camera module is required and the edge hardware is minimized.

The supporting infrastructure is fully local: an MQTT broker (Mosquitto) and the Home Assistant platform run as Docker containers on a home server. The system therefore operates with nothing more than the home modem, survives both international and domestic Internet outages, and even when the global network is available, no cloud service is required — user data stays inside the home network, preserving privacy and keeping network latency minimal. Four parallel user interfaces are provided: the on-device touch display (LVGL), a mobile web panel opened via QR scan, the Home Assistant web dashboard, and the Home Assistant companion app, all fed by a single source of state on the device.

Beyond vision, a lightweight behavior-learning agent (two logistic-regression models with offline pre-training at 94.2% / 93.6% validation accuracy and on-chip online SGD) learns user habits for lighting and fan control in a shadow-then-auto lifecycle; the door lock is deliberately never under model control. A three-layer face-based attendance system (device outbox → local FastAPI/SQLite server → Jalali-calendar Google Sheets archive and Bale messenger notifications) was also implemented and deployed. Reliability mechanisms — task watchdogs, a hardware-reset watchdog for the touch controller, and network-aware Wi-Fi/MQTT reconnection — were validated through end-to-end scenario testing, and the deployed system is documented with photographs and screen captures of the board, its display, the companion apps and the running services (chapter 11).

Keywords: Internet of Things, Edge AI, TinyML, ESP32-S3, Face Recognition, MQTT, Home Assistant, Docker, FreeRTOS, LVGL, Privacy, Local-first

Table of Contents

Note: after any edit, right-click the table and choose Update Field to refresh the page numbers.

List of Figures

2	Figure 1-1: Overall block diagram of the edge-processing smart-home system
14	Figure 4-1: Software layers of the system
17	Figure 5-1: Overall system architecture and local deployment
20	Figure 6-1: The face-recognition pipeline on the microcontroller
23	Figure 6-2: The shadow phase, promotion gate and online learning of the behavioral agent
24	Figure 7-1: The four parallel user-interaction channels of the system
27	Figure 8-1: The three-tier architecture of the attendance system
34	Figure 11-1: The deployed system on the workbench — board, display and companion app
36	Figure 11-2: The nine LVGL interface pages — screenshots from the board's real display
37	Figure 11-3: The system's web dashboard in its mobile view
38	Figure 11-4: The Home Assistant companion-app dashboard on a phone
39	Figure 11-5: The Bale bot — device control and event notifications
40	Figure 11-6: The system's running containers in Docker Desktop

List of Tables

8	Table 3-1: Functional requirements
9	Table 3-2: Non-functional requirements
12	Table 4-1: Hardware pin mapping
13	Table 4-2: Flash partitioning
14	Table 4-3: FreeRTOS tasks in the firmware
18	Table 5-1: Main MQTT topics
18	Table 5-2: Entities discovered in Home Assistant
21	Table 6-1: Input features of the behavioral learning agent
22	Table 6-2: Training parameters and agent performance
25	Table 7-1: Main web-server endpoints on the board
31	Table 10-1: Test scenarios and results

Chapter 1: Introduction

1.1 Problem Statement

Internet-of-Things (IoT) smart-home systems have spread rapidly in recent years; nevertheless, most common commercial solutions rest on two pillars: numerous dedicated hardware devices inside the house, and cloud services for processing, authentication and data storage. Relying on the cloud raises three serious problems. First, the user's privacy is handed over entirely to an external entity: sensitive data such as residents' face images leaves the house and is kept on servers that are outside the user's control. Second, the operation of the system becomes dependent on the stability of the Internet connection; every outage means losing control of — and intelligence in — the home. Third, the network latency of the cloud round-trip is unacceptable for latency-critical decisions such as unlocking a door with face recognition.

On the other hand, the intelligent part of such systems — machine vision in particular — has traditionally required powerful processors and a dedicated camera, which drives up cost, installation complexity and power consumption. The central question of this project can therefore be formulated as follows: is it possible to build a complete, secure and intelligent smart home with minimal edge hardware, in such a way that the artificial-intelligence processing (face recognition) runs entirely on the microcontroller itself, the camera sensor and the rich user interface are borrowed from hardware the user already owns (a smartphone with its camera and browser), and no data ever leaves the home network?

This project answers that question by designing and implementing a smart-home system based on the ESP32-S3 microcontroller, in which the user's face image is captured through the phone's browser (reached by scanning a QR code shown on the board's display), the neural-network inference runs locally on the microcontroller, and the entire supporting infrastructure (the MQTT message broker and the Home Assistant platform) is deployed locally on a home server using Docker. The result is a system that operates with nothing more than the home modem, shows no loss of functionality during international or even domestic Internet outages, and does not depend on any cloud service even when the Internet is available.

The overall block diagram of the system is shown in Figure 1-1.

All intelligence runs on the edge node — the phone only lends its camera, the servers stay local

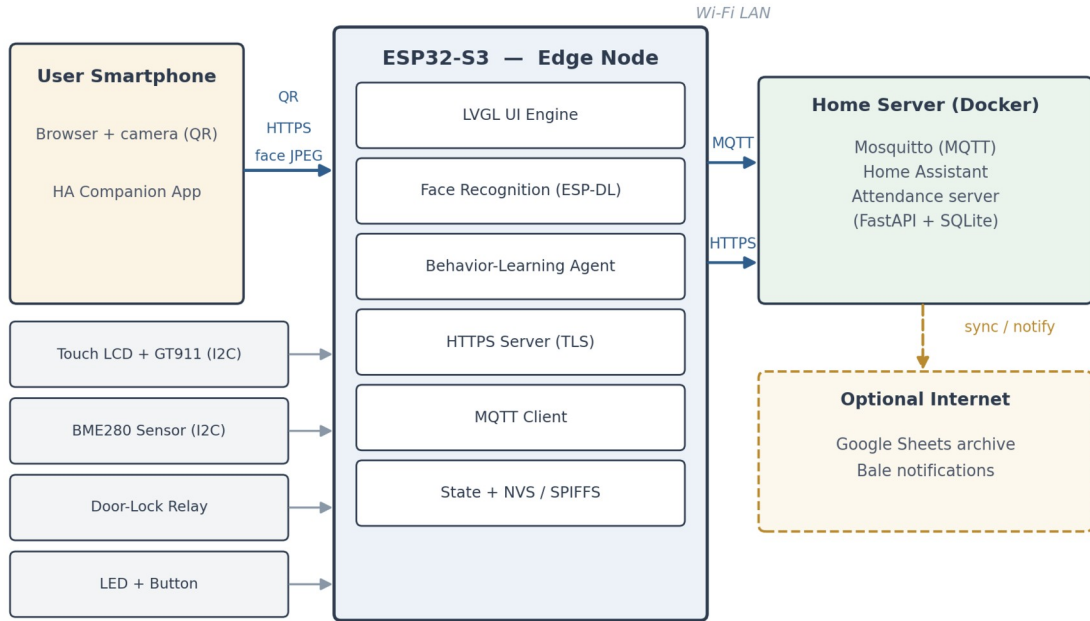


Figure 1-1: Overall block diagram of the edge-processing smart-home system

1.2 Project Objectives

The objectives of this project were defined as follows:

1. Implementing an ESP32-S3-based IoT node with a color touch display, a secure on-board HTTPS web server and stable connectivity to the home network;
2. Running the complete face-recognition pipeline (image decoding, face detection, feature-vector extraction and matching) on the microcontroller itself, without sending any image to an external service;
3. Using the user's phone camera as the image input through a mobile web page and a QR-code mechanism, instead of adding a dedicated camera module;
4. Deploying the local infrastructure — the MQTT broker (Mosquitto) and the Home Assistant platform — with Docker, and connecting the board to it through automatic service discovery;
5. Designing a multi-channel user interface: the touch display, a mobile web panel entered via QR scan, the Home Assistant web dashboard and the Home Assistant mobile app, all driven by a single source of state;

6. Developing a lightweight machine-learning agent that learns the user's behavioral pattern (light and fan control) by combining offline pre-training with on-chip online learning;
7. Implementing a face-based attendance system with a local server, a durable message queue, a Google Sheets archive with Jalali (Persian) dates and Bale messenger notifications;
8. Ensuring system reliability through runtime monitoring (watchdogs), network-aware reconnection logic and scenario-driven end-to-end tests.

1.3 Innovations and Key Features

Artificial intelligence on a constrained device: the most important axis of this project is moving intelligence to the edge. The entire face-recognition chain — from hardware JPEG decoding to feature extraction by a quantized neural network — runs on the microcontroller. This choice has two fundamental benefits: complete privacy (the image never leaves the board) and the near-total elimination of network latency in decision-making. At the same time, authentication reuses hardware the user already owns (the phone's camera and browser), keeping the edge hardware to a minimum.

Comprehensive user interaction: the system serves the user through four parallel channels — the on-board touch display (always available, even without a network), the mobile web panel entered via QR scan, the Home Assistant web dashboard and the Home Assistant mobile app. All channels are fed from a single source of state (the `app_state` module on the board) so that the state remains consistent everywhere.

Complete independence from the cloud: the MQTT broker and Home Assistant run as Docker containers on a home server. The system needs only the local network (the modem) and works fully during international and even domestic Internet outages; user data remains on the local network, which both guarantees privacy and minimizes network latency.

On-chip behavioral learning: the project's learning agent combines offline pre-training with online stochastic-gradient updates on the microcontroller itself, and operates in a shadow phase (predicting without acting) followed by an auto phase; the door lock is deliberately never under model control.

An attendance ecosystem: on top of the same face-recognition engine, a three-layer attendance system was implemented with a durable message queue on the board, a FastAPI server and an SQLite database, a Google Sheets archive with Jalali (Persian) dates, and Bale

messenger notifications — independent of Home Assistant and without storing any cloud credentials on the board.

A comprehensive engineering exercise: this project was deliberately designed to cover the full spectrum of challenges of a real engineering effort — electronic design and wiring, hardware and memory constraints, real-time multi-core operating-system programming, networking and the MQTT and HTTPS/TLS protocols, security, interface and user-experience design, and software engineering (modular architecture, Git version control and feature-driven development on separate branches).

1.4 Scope and Assumptions

The executable scope of the project is as follows: the edge hardware consists of an ESP32-S3-DevKitC-1 board with a 320×480 touch TFT display, a lock relay, a status light and a physical button, while the BME280 environmental sensor sits on the shared I2C bus. The system's image input is the user's phone camera; any fingerprint sensor or dedicated camera on the board is deferred to future work. The network infrastructure is the home LAN, and the services are deployed with Docker on a home computer/server; migration to a low-power always-on machine is on the development roadmap.

The main assumptions are: a home network with stable DHCP (with a reserved static address for the server in the practical deployment) is available; users reach the local network with smartphones carrying modern browsers (getUserMedia support); and the accuracy of the ESP-DL face engine with a 0.70 acceptance threshold is sufficient for a home environment. It is acknowledged that liveness detection is not implemented in the current version and is reported as a development path in the final chapter.

Chapter 2: Fundamentals and Technologies

2.1 The Internet of Things and Edge Processing

The Internet of Things (IoT) refers to a collection of physical devices that gather data from — or act upon — their physical environment through sensors and actuators, and exchange it over a network. In the traditional architecture, the intelligent part of such systems is moved to the cloud; however, a newer trend known as Edge AI, and its TinyML branch, runs machine-learning inference directly on low-power microcontrollers. The benefits of this approach include keeping sensitive data inside the device, removing the dependence on a continuous Internet connection, reducing decision latency, and lowering energy consumption and cost. This project is a complete practical instance of that architecture: both the vision inference (face recognition) and the online learning of the user's behavioral pattern run on the microcontroller itself.

2.2 The ESP32-S3 Microcontroller

The ESP32-S3 is an Espressif chip with a dual-core Xtensa LX7 running at up to 240 MHz, integrated 802.11b/g/n Wi-Fi, and a set of vector instructions that accelerate machine-learning computation. This project uses the ESP32-S3-DevKitC-1 development board with an ESP32-S3-WROOM-1-N16R8 module, which provides 16 MB of flash and 8 MB of octal SPI PSRAM clocked at 80 MHz. The generous PSRAM is the decisive factor in running the LVGL graphical interface, a TLS web server and the neural-network models on a single chip: for example, the decoded image buffer (153.6 KB) and the LVGL draw buffers (about 76 KB) live in PSRAM so that the 512 KB of internal memory remains free for stacks and sensitive network buffers. One important pinout constraint of this module is that GPIO35 through GPIO37 are unavailable (reserved for the octal PSRAM), which was accounted for in the circuit design.

2.3 FreeRTOS and Multi-Core Programming

The official ESP-IDF framework runs on top of FreeRTOS, a real-time operating system with a priority scheduler that executes on both cores of the ESP32-S3. Units of execution (tasks) are created with their own priority, stack and communication queues, and synchronization mechanisms such as queues, semaphores and mutexes are essential for sharing data between them. The main challenge in this project was the coexistence of tasks

with very different characters: a latency-sensitive graphical interface, heavy neural-network processing, a TLS web server and lightweight network tasks. The project's solution was to separate tasks with shallow queues (for example, a depth-1 queue for images entering the face engine) instead of sharing memory directly, and to register the heavy tasks with the task watchdog. This exercise practiced concurrency, deadlock and resource-starvation concepts in the form of a real problem.

2.4 The LVGL Graphics Library

LVGL is an open-source graphics library for embedded systems that provides modern widgets, touch support and partial rendering. Version 9.5 was used in this project, integrated with the official espressif/esp_lcd driver for the ST7796 display (an 8-bit parallel Intel 8080 interface clocked at 20 MHz) and the GT911 touch driver. Partial rendering is performed with two 320×60 buffers (about 76 KB), which were moved to PSRAM; this relocation is later (chapter 10) traced as the root cause of an out-of-memory error during the TLS handshake.

2.5 The ESP-DL Framework and Lightweight Machine Learning

ESP-DL is Espressif's official collection of components for running shallow, quantized neural networks on ESP chips. Two ready-made components from this framework were used in this project: HumanFaceDetect for face detection and HumanFaceRecognizer for extracting a face embedding. The models are pre-quantized so that they fit in the microcontroller's memory and compute budget. Alongside machine vision, the behavioral-learning part uses the simple yet effective logistic-regression model: with weights of only a few dozen bytes it can be stored on the chip and updated online by stochastic gradient descent (SGD) — a deliberate choice demonstrating the principle that intelligence does not always require heavy models.

2.6 The MQTT Protocol

MQTT is a lightweight messaging protocol built on the publish/subscribe pattern that operates through a central broker; thanks to its low overhead it is the de-facto standard in the IoT world. The key capabilities used in this project are: QoS levels (this system uses QoS=1 for state messages), retained messages, which keep the last state for newly subscribing clients, and the Last Will and Testament (LWT), which publishes an offline message on the

state topic when the board's connection drops abnormally. Separating components through hierarchical topics makes it possible to add new subscribers (the dashboard, the app, the attendance service) without modifying the board.

2.7 Home Assistant and Docker

Home Assistant is the most popular open-source home-automation platform, runs self-hosted, and through the MQTT Discovery mechanism automatically registers devices that publish configuration messages as entities. Docker is the containerization technology that runs each service together with its dependencies in an isolated package. The combination of the two is the backbone of the project's local infrastructure: Mosquitto and Home Assistant both run as containers on the home server, require no cloud service whatsoever, and can be moved easily to any other computer.

2.8 Secure Web (HTTPS/TLS)

Because the web pages and the face-authentication endpoints run directly on the board, the system's web server is deployed with the `esp_https_server` library and a self-signed certificate on TLS. On the first connection the user accepts the self-signed-certificate warning, after which all traffic (including the face image) is transferred encrypted. Modern browsers permit camera access (`getUserMedia`) only in a secure context over HTTPS; TLS is therefore both a security requirement and a technical prerequisite for the image input. The `mbedtls` memory tuning (a 16 KB input buffer for the image upload) and the network keep-alive configuration are practical lessons from this part, discussed further in chapter 10.

Chapter 3: Requirements Analysis and System Specification

3.1 Functional Requirements

The functional requirements of the system are given in Table 3-1.

Table 3-1: Functional requirements

Description	Requirement	Code
Periodic reading of temperature, humidity and pressure from the BME280 on the shared I2C bus	Sensor reading	FR-01
Publishing light, fan, lock and sensor state with QoS=1 and retained messages	Publishing state to MQTT	FR-02
Publishing Discovery configuration messages for 13 entities	Auto-discovery in Home Assistant	FR-03
Controlling light, fan and lock from the touch display, web panel, HA dashboard and HA app	Multi-channel control	FR-04
Receiving an image from the mobile browser, face matching on the board with a 0.70 threshold, and relay activation	Face unlock	FR-05
A numeric keypad on the touch display with visual feedback	PIN unlock	FR-06
Adding a new face through a six-digit, 3-minute token and a mobile web page; multi-sample grouping	Face enrollment	FR-07

Description	Requirement	Code
Entry/exit recording with a face via the QR code on the display; a durable queue and delivery to the local server	Attendance	FR-08
Predicting light and fan state with the on-chip model in the shadow and auto phases	Behavioral learning	FR-09
Changing passwords, Wi-Fi and MQTT broker settings from the display and the web pages	Settings management	FR-10

3.2 Non-Functional Requirements

The non-functional requirements of the system are given in Table 3-2.

Table 3-2: Non-functional requirements

Description	Requirement	Code
The face image is processed and freed only on the board; no data is sent to any cloud service	Privacy	NFR-01
Full operation on the local network alone; compatible with international and domestic Internet outages	Internet independence	NFR-02
The face-based lock decision loop stays inside the local network and never crosses the cloud	Low latency	NFR-03
Automatic recovery from Wi-Fi/MQTT outages, recovery from touch I2C lockups, and 15-second watchdog supervision	Reliability	NFR-04

Description	Requirement	Code
A TLS web server with a self-signed certificate, an HttpOnly/SameSite=Strict session cookie and time-limited tokens	Transport security	NFR-05
A modular architecture with a single source of state; new sensors/actuators without redesign	Extensibility	NFR-06
Periodic heap reporting every 30 seconds and structured logging for root-cause analysis	Debuggability	NFR-07

3.3 System Inputs and Outputs

The system inputs are: display touch events (the GT911 controller on I2C), the face image received from the user's phone browser (a JPEG of up to 300 KB), numeric PINs entered on the on-screen keypad, the physical button on the board, and the BME280 environmental sensor data (temperature, humidity, pressure).

The system outputs are: the graphical interface on the 320×480 display (dashboard, keypad, settings pages and QR codes), the door-lock relay command (active-high with a safe 8-second auto-relock), the light state, and the stream of MQTT messages to the local infrastructure, which feeds the Home Assistant dashboard, the mobile app and the attendance service.

3.4 Operational Scenarios

The main use-case scenarios of the system are as follows:

1. Door unlock with a face: the user scans the QR code on the display with their phone, the phone's web page captures the image and sends it to the board; the face engine on the board makes the decision and, on a match, energizes the lock relay for a safe interval.
2. Door unlock with a PIN: when no phone is available, the numeric keypad on the touch display accepts the lock PIN.

3. New face enrollment: the administrator issues a six-digit token from the settings page, enters the person's name on the mobile web page and captures several face samples; the samples are grouped under one name.
4. Attendance: the user presses the Attend button on the display, scans the 10-minute QR code, selects entry or exit and captures their face; the event goes to the local server and is reflected in the sheet and in Bale.
5. Home control from all four channels: light and fan from the display, the mobile web panel, the HA dashboard or the HA app; every change is reflected live in the other channels.
6. Behavioral learning: in the shadow phase the model reports and evaluates its predictions; after passing the promotion gate it enters the auto phase and adjusts the light and fan according to the behavioral pattern; every manual user action is executed immediately and recorded as a training sample.

Chapter 4: Hardware and Firmware Design

4.1 Hardware Architecture

The hardware core of the system is an ESP32-S3-DevKitC-1 development board (revision 1.1) with the N16R8 module, connected to a 320×480 touch TFT display driven by an ST7796 controller (an 8-bit Intel-8080 parallel interface), a GT911 capacitive touch controller, a BME280 environmental sensor, a lock relay, a status light and a physical button. The display and touch are driven through Espressif's official drivers, and the sensors and touch share one I2C bus (SDA = GPIO1, SCL = GPIO2), which requires external 4.7-kΩ pull-up resistors per the module specifications. The complete pin mapping is given in Table 4-1.

Table 4-1: Hardware pin mapping

Device / function	Direction	Pin
SDA / SCL of the shared GT911 and BME280 bus	I2C bus	GPIO1 / GPIO2
ST7796 display data bus and control signals	8-bit parallel output + control	GPIO4 – GPIO18 (bound)
Display backlight control (with a driver circuit)	Output	GPIO12
Light (LED)	Output	GPIO13
Physical button	Input	GPIO14
Door-lock relay (active-high, 8-second auto-relock)	Output	GPIO21
Interrupt pin (INT) of the GT911 touch controller	Input	GPIO41
Reset pin (RST) of the GT911 touch controller	Output	GPIO42
Internally reserved by the octal PSRAM (unusable)	—	GPIO35–37

Two important constraints were respected in the pin selection: avoiding the octal-PSRAM reserved pins, and keeping several pins free (GPIO39, GPIO40, GPIO47 and GPIO48) for future expansion such as driving a real electric lock.

4.2 Memory Partitioning

The partition table of the 16 MB flash is given in Table 4-2. The large app partition (8 MB) was chosen to accommodate the ESP-DL models, the web server, the graphical interface and the many modules simultaneously, and the SPIFFS partition stores the face-feature database.

Table 4-2: Flash partitioning

Purpose	Size	Partition
Storing settings: passwords, Wi-Fi, the MQTT broker, the learning-model weights, the attendance queue	24 KB	nvs
Radio calibration data	4 KB	phy
Main application (firmware)	8 MB	factory
Face-feature database (face_db)	1 MB	storage (SPIFFS)

Beyond the flash, the 8 MB of octal PSRAM serves as the large working area (the image buffer, the render buffers and the mbedTLS auxiliary memory), while the internal memory is reserved for stacks and latency-sensitive buffers.

4.3 Modular Firmware Structure

The firmware is developed with ESP-IDF version 6.0.2 in C and is organized as independent modules: Wi-Fi management (wifi_manager), MQTT management (mqtt_manager), a secure web server (http_server), the face engine (face_recognition), the face database (face_db), enrollment tokens (enroll_token), attendance (attendance), the learning agent (ml_agent), the graphical interfaces (display/*), lock management (lock), the sensor (bme280/sensor_task), time synchronization (time_sync) and virtual devices (virtual_devices). The software layering is shown in Figure 4-1.

Software stack (bottom = hardware, top = user-facing services)

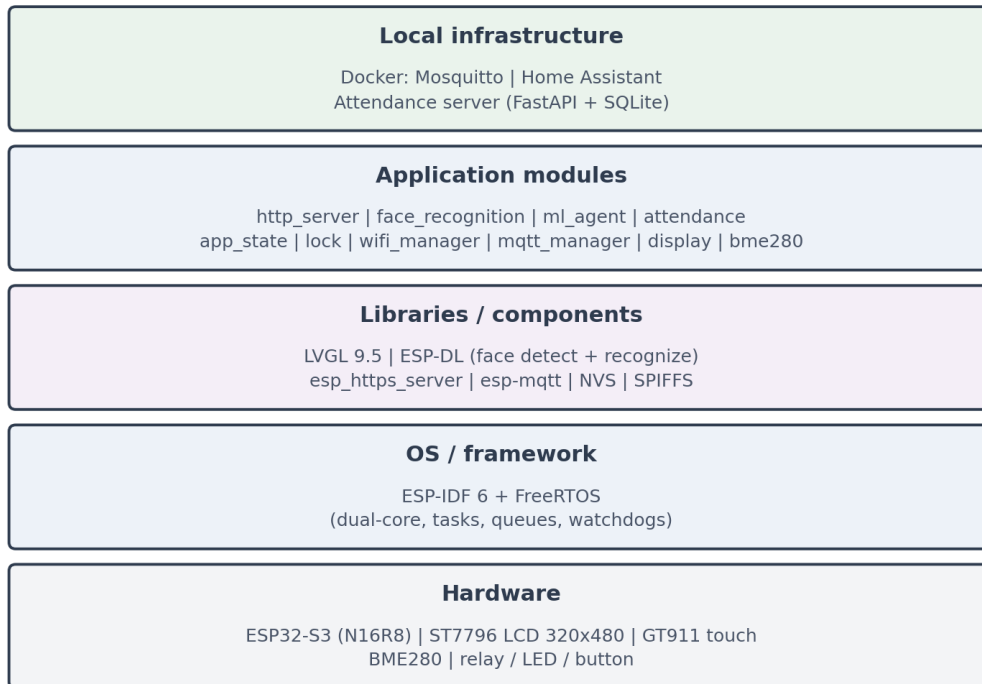


Figure 4-1: Software layers of the system

The key design principle is the single source of state: the `app_state` module is the only authority on the state of the light, fan, lock and sensors, and every change from any channel (touch, web, MQTT, button or model) passes through this layer. As a result, the display, the web server and MQTT always present a consistent picture of the system, and the learning agent's training path is also fed from a single point.

4.4 FreeRTOS Tasks and Synchronization

The main firmware tasks and their configuration are given in Table 4-3.

Table 4-3: FreeRTOS tasks in the firmware

Role	Period / pattern	Priority	Stack	Task
Rendering the UI and processing touch	2 ms tick	2	8 KB	LVGL

Role	Period / pattern	Priority	Stack	Task
Serialized face detection and matching	Event-driven (depth-1 queue)	3	16 KB	face_worker
Reading the BME280 and publishing state	Every 5 s	2	3 KB	sensor_task
Inference and behavioral learning	Every 30 s	3	4 KB	ml_agent
Sending attendance events from the NVS queue	Event-driven	2	8 KB	attendance
Network-aware automatic reconnection	Every 20 s	2	—	wifi_retry
Hardware reset of the GT911 on lockup	Supervisory	—	—	touch_wdt
Serving HTTPS requests	Event-driven	—	—	httpd (TLS)

Synchronization is performed with minimal shared memory: input images are handed to `face_worker` through a depth-1 queue so that model execution is serialized and memory freeing is deterministic; access to the shared I2C bus is protected with a mutex; and device-state changes are made only through `app_state`.

4.5 Reliability and Error Handling

The reliability of the system rests on four mechanisms. First, the task watchdog (TWDT) is active with a 15-second threshold and panic mode enabled, and the heavy `face_worker` task feeds it periodically (at most every 5 seconds) so that a long neural-network execution does not cause repeated resets. Second, the touch watchdog monitors the health of the GT911 and, on an interrupt gap longer than 6 seconds or consecutive I2C errors, recovers the touch controller with a hardware reset sequence (RST and INT). Third, network-aware

reconnection logic: the Wi-Fi manager tries the list of known networks in most-recently-used order with an attempt quota, and the MQTT manager — with a four-state machine and five consecutive attempts (every 5 seconds) before each connection — probes the broker address with a temporary client. Fourth, memory monitoring: every 30 seconds the heap state and the largest free internal block are logged so that leaks or corruption surface early.

4.6 Settings and Persistent Data Storage

All user settings are kept in NVS: the lock PIN and the settings password (with a minimum-length check), the Wi-Fi network list ordered by most recent use, the MQTT broker address (enterable as an IP or a domain name for DDNS compatibility), the attendance configuration, active tokens, and the versioned weight block of the learning model (about 90 bytes), which is saved every 30 seconds when changed and safely rolled back to the pre-trained weights on a version mismatch. The face-feature database lives in the SPIFFS partition, and its metadata (the identifier-to-name mapping and the sample counters) is managed in NVS.

Chapter 5: Communication Architecture and Local Deployment

5.1 A Cloud-Free, Local-First Approach

The communication architecture of the project is built on the everything-local principle: the MQTT broker (Mosquitto) and the Home Assistant platform run as Docker containers on a home server, and the board connects to them over the home Wi-Fi (Figure 5-1). This deployment needs nothing more than the home modem; during international Internet outages — and even a complete domestic outage — every feature, from face unlock to lighting control, the dashboard and attendance, keeps working without interruption. When the Internet is available, the optional external services (the Google Sheets archive and Bale notifications) are fed through the local server, but they are never a precondition for operation and no critical data depends on the outside. A second benefit of this architecture is very low latency: the complete face-decision loop stays inside the local network.

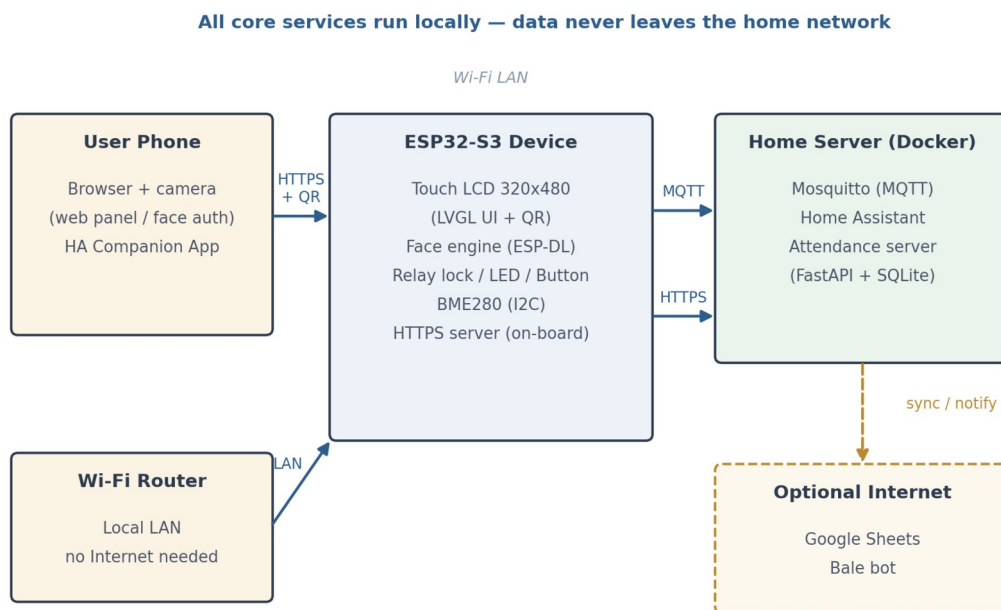


Figure 5-1: Overall system architecture and local deployment

5.2 Topic Structure and Message Format

All system messages are exchanged under the smarthome/ prefix with QoS=1 and retained mode for state topics. Table 5-1 lists the main topics.

Table 5-1: Main MQTT topics

Message format	Communication path	Topic
online / offline (will message)	board → all (LWT)	smarthome/status
ON / OFF	board ↔ HA and web	smarthome/light/state set
ON / OFF	board ↔ HA and web	smarthome/fan/state set
LOCKED / UNLOCKED	board → all	smarthome/lock/state
JSON: temperature, humidity, pressure	board → HA	smarthome/sensor/state
JSON: access-event type	board → HA	smarthome/access/state
HOME / AWAY	board → HA	smarthome/presence/state
AUTO / SHADOW	board ↔ HA	smarthome/ml/mode mode/set
JSON: probabilities, accuracies and counters	board → HA	smarthome/ml/stats

5.3 Integration with Home Assistant

On connection, the board publishes the MQTT Discovery configuration messages for 13 entities, and Home Assistant registers them automatically (Table 5-2). HA connects to the broker through the Docker service name rather than an IP address, making the deployment independent of network layout. An important security decision was to report the door lock as a binary state sensor instead of a lock entity, so that before the broker is hardened there is no lock command surface at the HA level.

Table 5-2: Entities discovered in Home Assistant

Entities	Group
Light, fan, and the model auto-mode switch	Actuators
Temperature, humidity, pressure, light intensity (lux)	Environmental sensors
Lock state (binary_sensor), presence (binary_sensor), access event (event)	State and security

Entities	Group
Agent mode, agent statistics (with JSON attributes), two prediction-probability sensors for light and fan	Behavioral learning

5.4 Connection-State Management

The connection lifecycle is tied to the network state: when Wi-Fi comes up, the MQTT connection starts, and when Wi-Fi drops, the MQTT connection is actively torn down so that pointless retries and pending queues do not accumulate. With every abnormal disconnection, the will message (LWT) publishes offline on smarthome/status, and retained messages ensure that the dashboard and the app see the last correct state immediately after connecting. The broker is probed with a temporary client before being saved into the settings, so that an address error surfaces at the moment of entry.

Chapter 6: Artificial Intelligence on the Edge

6.1 On-Device Face-Processing Architecture

The intelligent heart of the project is the complete face-recognition pipeline that runs entirely on the ESP32-S3 (Figure 6-1). A JPEG image (up to 300 KB) is sent from the user's phone browser to the board over HTTPS; the `esp_jpeg` module decodes it with hardware acceleration into an RGB565 buffer of 320×240 (153.6 KB in PSRAM); the face-detection model (HumanFaceDetect from the ESP-DL framework) then yields the face coordinates, and the recognition model (HumanFaceRecognizer) extracts the image's feature vector. This vector is compared with the enrolled vectors in the face database (the SPIFFS partition), and if the similarity exceeds the 0.70 threshold the identity is confirmed and the lock command or the attendance event is issued.

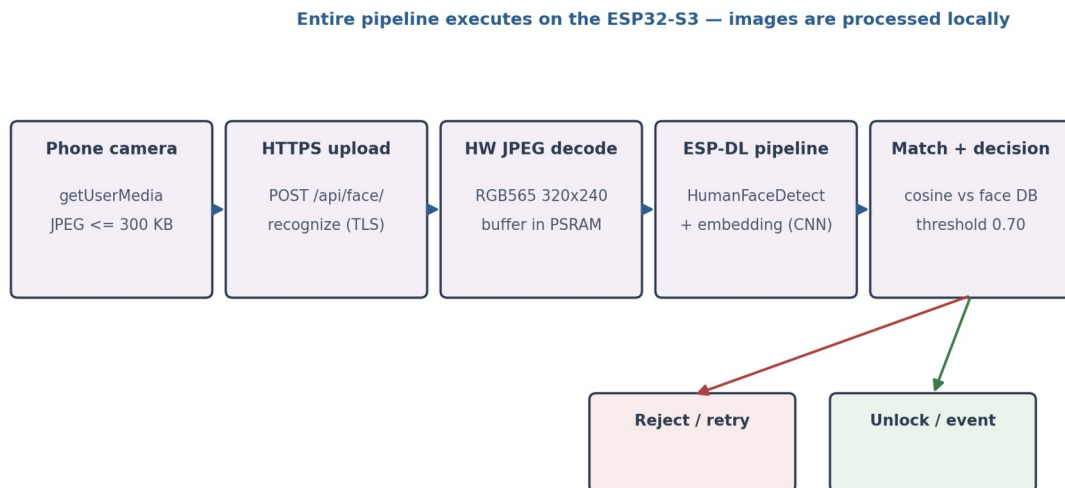


Figure 6-1: The face-recognition pipeline on the microcontroller

Running this pipeline on a microcontroller dictated three important engineering decisions. First, a depth-1 input queue: concurrent requests are queued and model execution is serialized to prevent memory interference and instability. Second, placing the buffers in PSRAM: the decode buffer and the model's large variables live in PSRAM, leaving the internal memory free for stacks and mbedTLS. Third, non-blocking hand-off: the web server does not wait for processing to finish; the `face_worker` task applies the result independently, keeping the UI responsive.

6.2 Face Enrollment and Sample Management

The enrollment process was designed with the simultaneous goals of simplicity and security: the administrator creates an enrollment link from the settings page (protected by a password and a 10-minute session); the server generates a six-digit token valid for 3 minutes, shareable as an `https://IP/enroll?token=...` link. The user enters the person's name on the mobile web page and captures the face image (POST `/api/face/enroll`). The token is multi-use until it expires so that several samples of one person can be captured. A person's multiple samples are grouped under one name in the `face_db` layer (with a sample counter), and this multi-sample enrollment measurably improves matching under varying light and pose. The 0.70 acceptance threshold was kept after practical experiments with multi-sample enrollment, and further calibration was deliberately postponed to later versions.

6.3 The User Behavioral-Learning Agent

Alongside machine vision, the project includes a lightweight learning agent that learns the user's behavioral pattern for light and fan control. The model is two simple logistic regressions operating on a 12-dimensional context-only feature vector (Table 6-1): hourly characteristics as sinusoidal harmonics, the Thursday/Friday holiday state, resident presence, and the normalized light intensity, temperature and humidity. The own-current-state feature was deliberately removed: in the first version its presence left the model passive in auto mode despite 91% accuracy (inert behavior), and removing it — together with adding the third harmonic — raised accuracy from 88% to 94%.

Table 6-1: Input features of the behavioral learning agent

Description	Count	Feature group
The model's constant term	1	Bias
Three sin/cos harmonics covering the daily pattern	6	Hourly harmonics
Thursday and Friday	2	Holidays
HOME after each successful unlock (at least 45 min)	1	Resident presence
Normalized light intensity and temperature (humidity in the vector)	2	Environmental context

Training is hybrid: offline pre-training on a 30-day synthetic dataset (8,640 samples in 5-minute steps, generated from the user's policy with an intentional 4–5% label noise), followed by online learning with stochastic gradient descent (learning rate 0.08 and weight clipping) on the chip itself. The parameters are given in Table 6-2.

Table 6-2: Training parameters and agent performance

Value	Parameter
30 days, 8,640 samples (5-minute steps)	Pre-training dataset
Full-batch gradient, 1,500 iterations, with L2	Offline optimization
The final 6 days of data	Validation
SGD at rate 0.08, weight clipping (± 8)	Online learning
NVS (a versioned ~90-byte block, every 30 s)	Weight storage
Every 30 seconds	Decision period
$p \geq 0.75$ on, $p \leq 0.25$ off	Action thresholds
Light 94.2% — Fan 93.6% (Bayes cap $\approx 95\%$)	Final accuracy (validation)

The agent operates in two phases (Figure 6-2): in the shadow phase the model predicts every 30 seconds, reports to MQTT and is evaluated, but takes no action; after at least 15 correct decisions in a 20-decision window it is automatically promoted to the auto phase. In the auto phase the model's own actions are not used for training, whereas every manual user action (from any channel) is executed immediately and recorded as a training sample. The definitive security rule is that the door lock is outside model control.

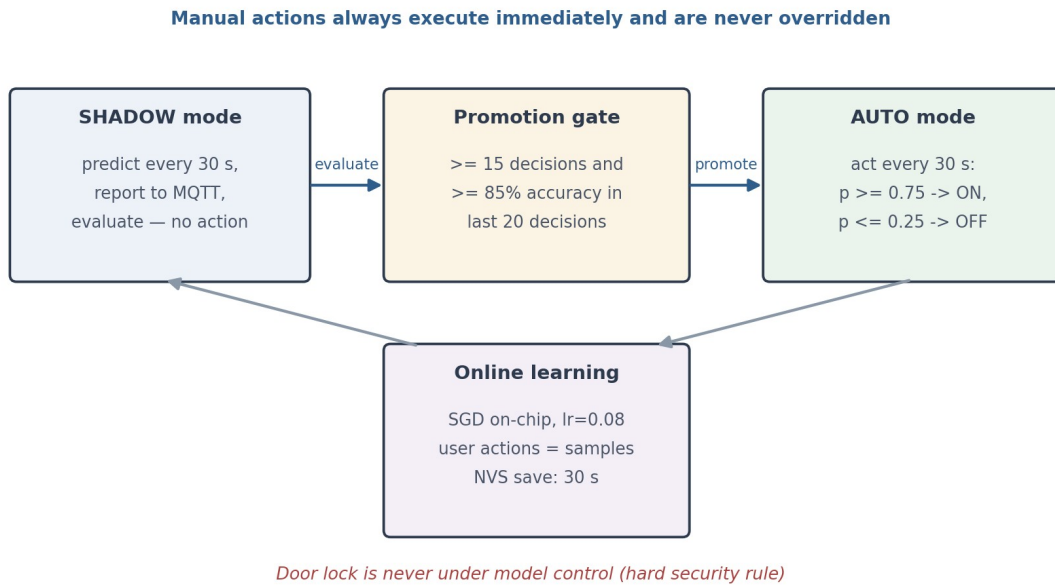


Figure 6-2: The shadow phase, promotion gate and online learning of the behavioral agent

6.4 Memory and Resource Considerations

Running two vision models, a graphical interface and a TLS web server simultaneously on one chip turned resource management into a primary design problem: the ESP-DL models are pre-quantized to fit the limited flash and memory; the behavioral agent's weights total a few dozen bytes; the image buffer and the render buffers were moved to PSRAM; and periodic monitoring of the largest free internal block serves as an early memory-corruption alarm. The result is a system that delivers meaningful intelligence with tens of dollars of hardware and no companion processor.

Chapter 7: Multi-Channel User Interfaces

7.1 The Multi-Channel Architecture

One of the design pillars of this project is comprehensive interaction coverage: the user must be able to reach the system in any situation — at the door, in another room, or away from home. Four parallel channels were therefore designed (Figure 7-1): the on-board touch display, which is always available even without a network; the mobile web panel, opened by scanning the QR code shown on the display, which uses the phone's camera as the authentication sensor; the Home Assistant web dashboard for management from a computer; and the Home Assistant mobile app, which brings the same entities onto the phone. All channels are fed from the single source of state so that the experience remains unified.

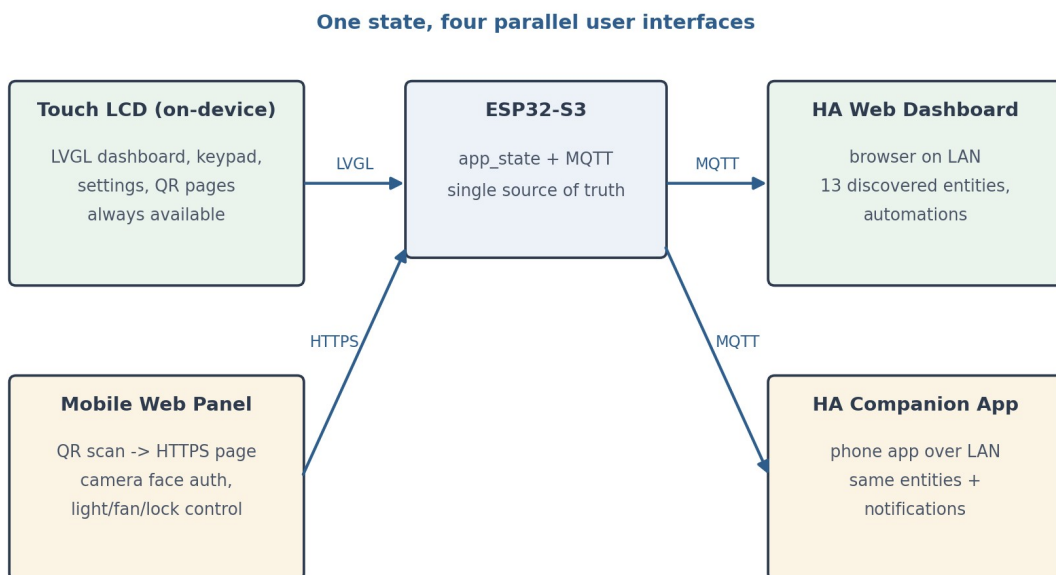


Figure 7-1: The four parallel user-interaction channels of the system

7.2 The Touch Display (LVGL)

The on-board interface built with LVGL 9.5 consists of six pages: the main dashboard (Wi-Fi state and IP address with a QR code, MQTT state with a real switch, the attendance button, light and fan controls, the unlock button, the model-mode button that toggles between auto/shadow on a short tap and shows details on a long press, and three 96×96 sensor cards for temperature/humidity/pressure); the lock's numeric keypad with visual feedback; the Wi-Fi setup page (scanning networks, highlighting recent networks, a password keyboard with a show/hide key); the settings page (changing the lock PIN and the settings password, the

list of enrolled faces with deletion); the MQTT broker configuration page; and the full-screen attendance QR overlay with a countdown. The page design focuses on responsiveness and readability from a distance, and sensitive pages return to the dashboard when touch is idle.

7.3 The Mobile Web Panel with QR Entry

The secure web server on the board (esp_https_server) serves the mobile panel pages directly; the user scans the display's QR code to reach https://IP/... and, after accepting the self-signed certificate, the panel opens. The three key pages are: the face-recognition page (/recognize), which opens the camera with getUserMedia, crops the central frame to 320×240 and sends it to the board as a JPEG; the token-protected enrollment page (/enroll); and the token-protected attendance page (/attendance) with a 10-minute token. Light and fan control and PIN unlocking are also possible from the panel. The main web-server endpoints are listed in Table 7-1.

Table 7-1: Main web-server endpoints on the board

Role	Path
Web dashboard panel (light, fan and lock control; sensor readouts)	/
Mobile face-recognition page (entered via the display's QR code)	/recognize
Enrollment page protected by a 3-minute token	/enroll?token=...
Attendance page protected by a 10-minute token	/attendance?token=...
Settings login page (creates a session)	/password
POST of a JPEG image for face matching	/api/face/recognize
POST enrolling a new face sample	/api/face/enroll?token=&name=
POST recording entry/exit with an image and token	/api/attendance
POST light and fan commands	/api/light/set /api/fan/set
POST unlocking with a PIN (with a deliberate 1-second delay on error)	/api/lock/unlock
POST entering settings and issuing the session cookie	/api/settings/unlock

Role	Path
POST management operations (the enrollment link, etc.)	/api/settings

The settings session is kept as a random 16-byte token in the board's memory and delivered in a shs cookie with HttpOnly, Secure and SameSite=Strict flags and a 10-minute expiry; only one active session is permitted at any moment. Binding the basic control endpoints (light/fan/sensors) to HA was deliberately kept loose so that the home remains controllable even without the server — a conscious decision for resilience.

7.4 The Home Assistant Dashboard and Mobile App

Thanks to MQTT auto-discovery, all 13 entities are immediately available in Home Assistant and appear in both the web dashboard and the mobile app without any extra configuration. The user can control the light and fan from anywhere in the house, view access events and presence state, monitor the behavioral agent's statistics and probabilities, and switch the auto/shadow mode. This channel provides a complete management panel without a single additional line of code on the board — a clear demonstration of the value of a standards-based (MQTT) architecture.

7.5 User Experience and UI/UX Lessons

Several user-experience patterns recurred in this project: short-path entry via QR instead of typing addresses and passwords; countdowns and automatic token renewal to prevent expired QR codes; immediate visual feedback for every action (from the MQTT state change to the attendance confirmation); graceful degradation on failure (when the network is gone, the display still manages the lock and lights); and aligning the mobile web pages' language with Persian. This part was in effect a practical exercise in the principle that, in embedded systems, a good user interface affects the adoption of the system as much as the correctness of its logic does.

Chapter 8: The Face-Based Attendance System

8.1 The Three-Tier Architecture

On top of the same face-recognition engine, a complete attendance system was implemented that is deliberately independent of Home Assistant and MQTT, organized in three tiers (Figure 8-1): the board tier (QR generation, face matching, the durable queue), the local server tier (FastAPI in Docker with an SQLite database) and the optional delivery tier (the Google Sheets archive and Bale notifications over the Internet). Credential separation is enforced: the board holds no Google keys or Bale tokens, and communication with the server uses only a shared secret.

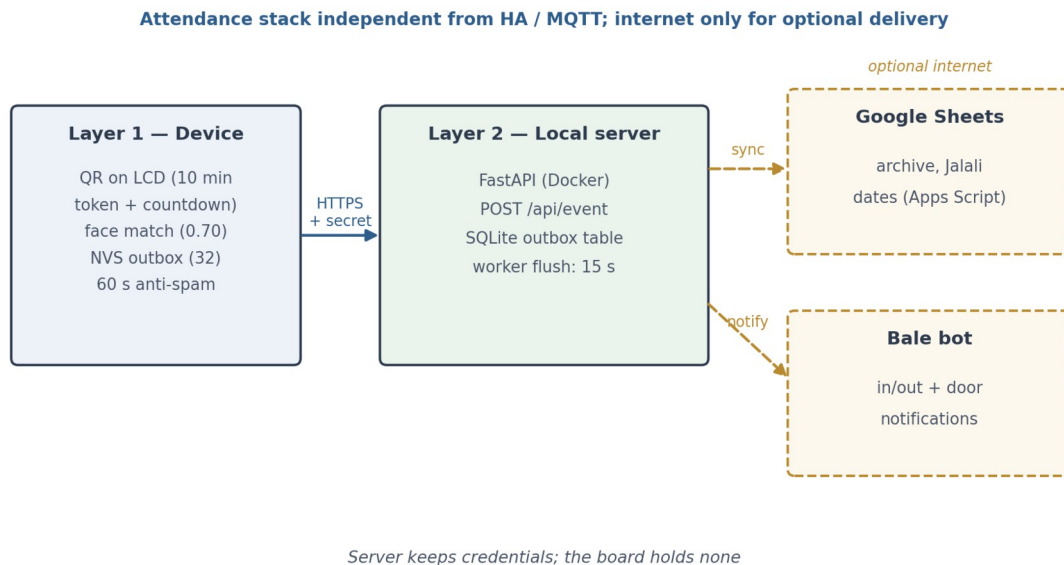


Figure 8-1: The three-tier architecture of the attendance system

8.2 The Attendance Recording Flow on the Board

The user taps the Attend button on the display's dashboard; the display shows the QR code of the `https://IP/attendance?token=...` address with a 10-minute validity and renews it automatically with a countdown. The user scans the QR code with their phone, selects Entry or Exit on the Persian page and captures their face with the phone's camera; the board matches the face with the same chapter-6 pipeline, extracts the person's name from the face database and creates an `attendance_in` or `attendance_out` event with the device's timestamp. To prevent duplicate records, the same person recording the same event type within a 60-

second window is suppressed. Door-unlock events — by face and by PIN — are also sent to the server for reporting.

8.3 The Durable Queue and the Local Server

Because losing a record is unacceptable, events are stored in a 32-slot queue in NVS before being sent, and an independent sender task delivers them to the server reliably; if the network or the server is unavailable, the records remain and are cleared in order once connectivity returns (attendance sending requires device timekeeping, and is rejected with an explicit error without NTP synchronization). The server — a FastAPI service in a Docker container with the `/api/event`, `/health` and `/stats` endpoints — records every event in an SQLite table. Acting as an outbox, this table carries partial indexes: a background worker picks up unsynchronized records every 15 seconds and pushes them to the integrations; even during an Internet outage the data stays intact in SQLite and the synchronization completes later.

8.4 Notifications and Archiving

The server has two optional integrations: the Bale messenger (with a Telegram-compatible API), which sends notifications for entry, exit, face-based unlocking and PIN unlocking; and a Google Sheets archive through an Apps Script webhook, to which each record is appended automatically. Dates in the sheet and in the Bale messages use the Jalali (Persian) calendar (an independent, dependency-free conversion in the `jdate` module), while timestamps in SQLite are kept in the Gregorian calendar. The face-similarity value is deliberately kept only in SQLite and is not transferred to the sheet, so that the management archive stays clean. Implementing this layer in full was a practical example of designing multi-component systems with a durable queue and synchronization in the face of unavailability.

Chapter 9: Security and Privacy

9.1 Privacy of Face Data

The most sensitive data in this system is the residents' face imagery, and the project's architecture ensures that this data never leaves the device's domain or the local network: the image is sent directly to the board itself, decoded and processed on that same chip, and the only outputs of the processing are a decision and a feature vector in the face database — no raw image, and no dependence on a cloud face-recognition service. Consequently, even under full network eavesdropping, no usable image content is available to an attacker. This characteristic — intelligence on a constrained device instead of cloud hardware and intelligence — is the project's central focus.

9.2 Web-Layer Security

All pages and endpoints are served over TLS with the board's self-signed certificate; this choice, beyond encryption, is the technical prerequisite for browser camera access. Sensitive flows carry two-stage authentication: the settings session (admin password → a secure 10-minute single-session cookie) and time-limited tokens for enrollment (3 minutes) and attendance (10 minutes), which are transmitted through QR codes and have a limited scope. PIN validation embeds a deliberate 1-second delay on wrong answers to neutralize rapid guessing. Passwords are stored in NVS with a minimum-length check.

9.3 Door-Lock Security

The door lock is protected by three hard rules. First, unlocking is possible only through verified paths (a face above threshold, the keypad PIN, the web-panel PIN and the physical button), and there is no lock command path from Home Assistant at all; the lock state is reported only as a binary sensor. Second, the learning agent is structurally barred from controlling the lock. Third, the lock relay is designed with an independent failsafe: even if the firmware dies, the lock automatically re-engages within 8 seconds at most.

9.4 Current Security Boundaries and the Hardening Path

In the current version, the Mosquitto broker runs without authentication for the simplicity of a home deployment, and this security boundary was chosen knowingly as the first step; fully hardening the broker (user/password authentication and TLS) and migrating the web

authentication mechanism to the WebAuthn standard on top of the existing tokens sit at the top of the project's security roadmap. This separation of the lock command path from the other capabilities was designed from day one so that those upgrades are possible without touching the board, by changing only the infrastructure.

Chapter 10: Tests, Results and Evaluation

10.1 Test Method

The system was evaluated at three levels: unit testing of components (bringing up each driver and module in isolation and inspecting logs), integration testing (message exchange between the board, the MQTT broker and Home Assistant using MQTT client tools), and scenario-driven end-to-end testing (running complete application cycles as a real user). Development followed a feature-driven flow on separate Git branches, with each feature merged only after on-board testing; every capability was therefore measured under real conditions before entering the main line.

10.2 Test Scenarios

The main test scenarios and their results are given in Table 10-1.

Table 10-1: Test scenarios and results

Result	Expectation	Condition / input	Scenario
Pass	Lock opens and an access event is raised	Enrolled face, different light and pose	Face unlock
Pass	No unlock; retry possible	Unenrolled person	Rejecting an unknown face
Pass	Unlock / error with a 1-second delay	Correct / wrong PIN	PIN unlock
Pass	Grouping under one name and improved matching	Valid token, 3 samples	Multi-sample enrollment
Pass	Request rejected and a new token issued	QR after 3 / 10 minutes	Token expiry
Pass	Recorded on the server, the sheet and a Bale notification	Enrolled face + token	Attendance entry/exit
Pass	The second event suppressed	Two records < 60 s apart	Attendance de-duplication
Pass	Records retained and sent after reconnection	Server outage and recovery	Durable queue
Pass	Most recent network selected and joined	Radio toggled	Wi-Fi reconnection

Result	Expectation	Condition / input	Scenario
Pass	5 consecutive attempts and subscriptions restored	Broker reset	MQTT reconnection
Pass	All local features keep working	Internet fully cut	Behavior during an Internet outage
Pass	Threshold crossed, shadow $\rightarrow$ auto transition	20-decision window	Behavioral-agent promotion
Pass	No unintended resets (TWDT supervision)	Continuous operation	Long-term stability
Pass	Hardware reset and continued operation	I2C lockup of the GT911	Touch recovery

10.3 Behavioral-Agent Results

The final model reached 94.2% accuracy for the light and 93.6% for the fan on the validation data of the final six days; the theoretical (Bayes) cap of the problem, given the intentional 4–5% label noise, is about 95%, so the model has effectively reached the problem's boundary. Three reference behavioral scenarios were also re-tested on the deployed firmware: night + presence + darkness ($p \approx 0.99 \rightarrow$ light on), noon with the light on ($p \leq 0.05 \rightarrow$ light off), and noon with the light off ($p \approx 0.07 \rightarrow$ no action). The automatic promotion from the shadow phase to auto — after at least 15 correct decisions in the 20-decision window — was observed in the practical test.

10.4 Engineering Challenges Resolved

Along the way, several real problems were root-caused and fixed, each carrying its own engineering lesson:

1. Out of memory during the TLS handshake after adding web graphics: root-cause analysis showed the LVGL render buffers occupying internal RAM; moving the buffers to PSRAM together with network keep-alive tuning (idle 30 / interval 5 / count 3), enabling `lru_purge` and 16 lwIP sockets resolved the problem permanently.
2. An I2C bus lockup of the GT911 touch controller: a mutex leak on the error path led to deadlock; fixing the lock-release path and adding a touch watchdog with a hardware reset sequence (RST and INT) made the system self-healing against this failure.

3. Watchdog panic loops in the face task: a long blocking wait for new work starved the sensing task; periodic TWDT feeding (at most every 5 seconds) in the wait loop and a 20-second timeout for receiving the image body guaranteed stability.
4. A passive behavioral model despite high accuracy: using the own-device-state feature caused inertia and inaction in auto mode; redesigning the feature vector to be context-only solved the problem at its root (chapter 6).

10.5 Overall Evaluation

The overall evaluation shows that the system met its design goals: the complete face-unlock loop runs inside the local network and on the microcontroller; the local MQTT and Home Assistant infrastructure works without interruption through Internet outages; the four user channels present a consistent picture of the state; the attendance system with its durable queue and Jalali calendar has operated in a real environment; and the firmware has remained stable in long-running operation without unintended resets. The reliability mechanisms put in place (watchdogs, network-aware reconnection and memory monitoring) provided valuable practical experience in engineering scalable embedded systems.

Chapter 11: Running on Real Hardware

11.1 The Deployment Environment and Real Equipment

The final validation of any embedded system is its deployment and operation in a real environment. This chapter presents photographic documentation of the system as deployed: from the ESP32-S3 board and its touch display on the workbench to the Home Assistant companion app on a phone, the Bale bot, the Google Sheets archive and the running Docker containers. Full recordings of the live tests are available in the project repository.

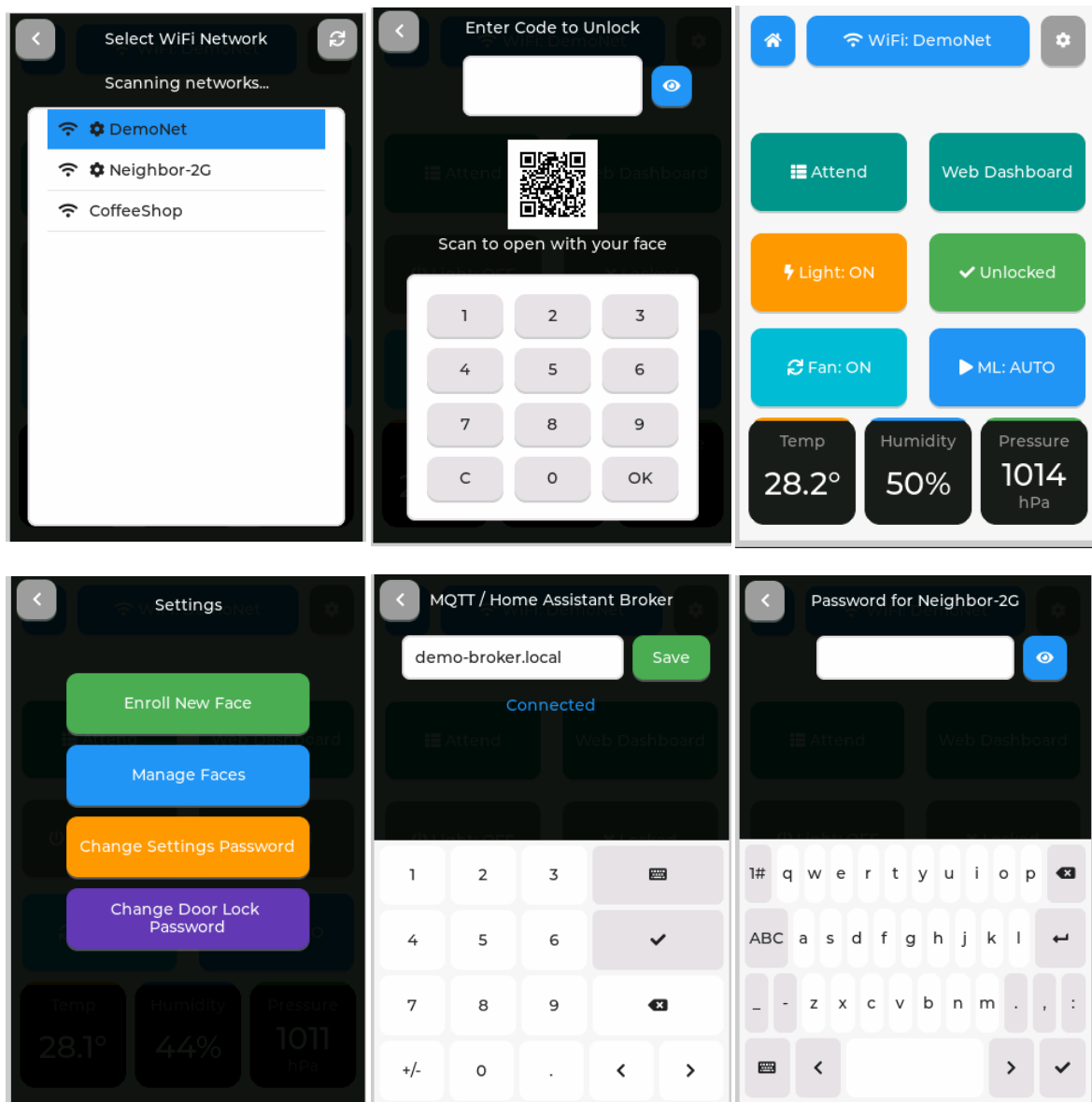
Figure 11-1 shows the final state of the workbench: the ESP32-S3-DevKitC-1 board (top) is connected to the 320×480 display through FPC adapters, the input/output circuitry (the lock relay, status LEDs and the button) is wired on a breadboard, and next to it the Home Assistant companion app on a phone mirrors the live state of the system.



Figure 11-1: The deployed system on the workbench — board, display and companion app

11.2 The Real Interface on the Display

The nine LVGL interface pages (the main dashboard, door unlock, Wi-Fi scanning and joining, PIN entry, MQTT configuration, settings, the faces list and the two QR pages for attendance and web access) were captured directly from the board's real display and are shown in Figure 11-2. These captures demonstrate that the entire setup cycle — from joining the network and configuring the broker to managing faces and passwords — can be completed without a computer, using only the touch display.



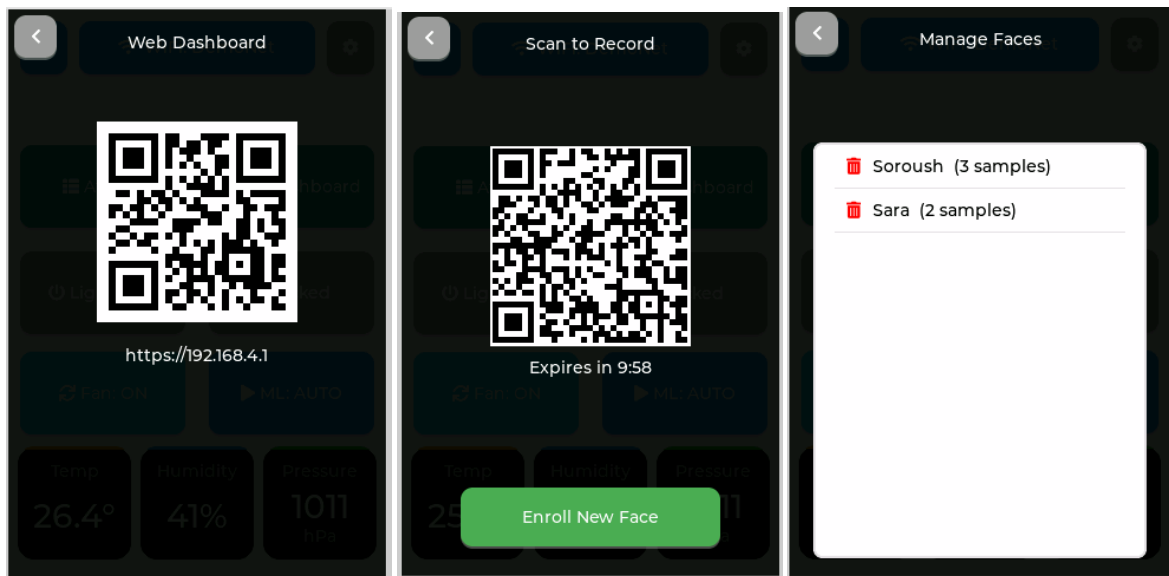


Figure 11-2: The nine LVGL interface pages — screenshots from the board's real display

11.3 The User Channels in Practice

Figure 11-3 shows the system's web dashboard in its mobile view: the light and fan switches, door lock with PIN and face unlock, live environment state and the learning agent's mode (SHADOW/AUTO). The dashboard is served by the local server and is reached by scanning the QR code shown on the board.

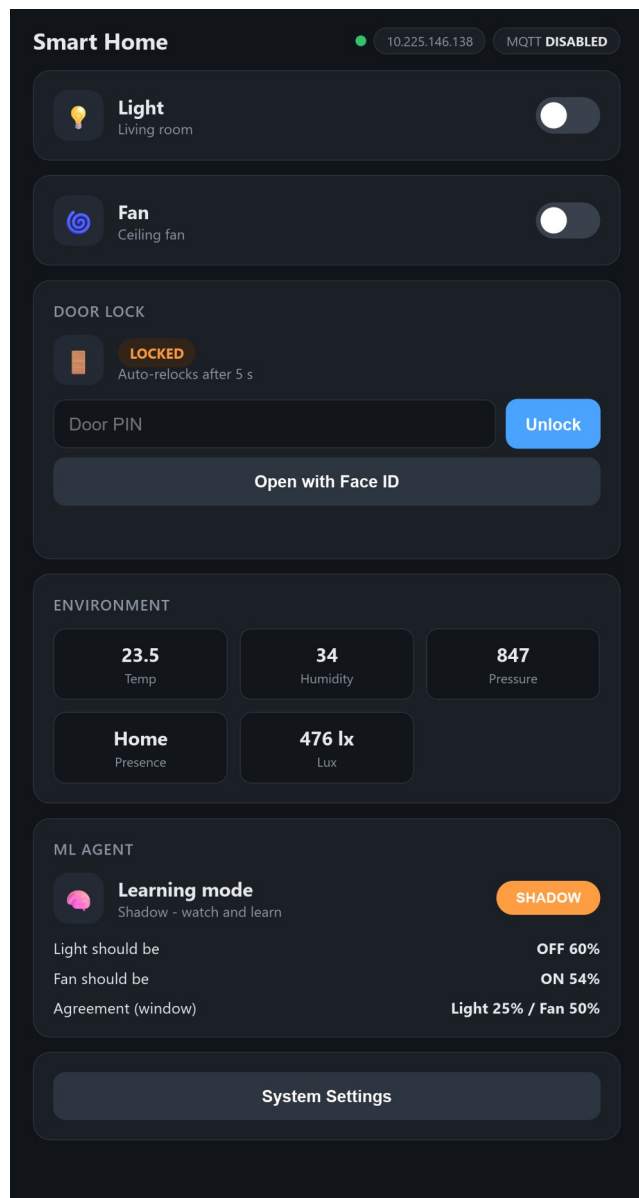


Figure 11-3: The system's web dashboard in its mobile view

Figure 11-4 shows the Home Assistant companion app's dashboard on a real phone: environment cards, device controls, access state and the four real automations defined in `automations.yaml` — all states synchronized bidirectionally over MQTT.

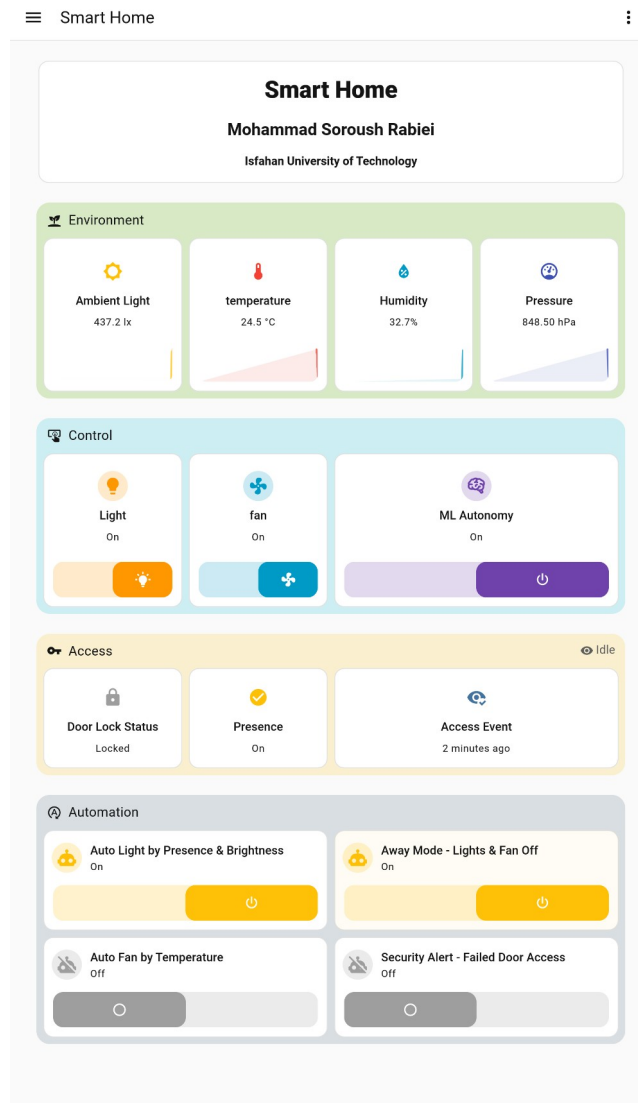


Figure 11-4: The Home Assistant companion-app dashboard on a phone

Figure 11-5 shows the Bale bot in operation: the Persian command menu, light and fan control through the reply keyboard, door opening with a 2-minute one-time password, and instant notifications of events (the door opening, residents entering and leaving).



Figure 11-5: The Bale bot — device control and event notifications

11.4 Service Deployment and Archiving

Figure 11-6 shows Docker Desktop on the home server; four lightweight containers (the attendance service, Mosquitto, Home Assistant and the Bale bot) with a combined memory footprint of a few hundred megabytes form the platform's entire infrastructure.

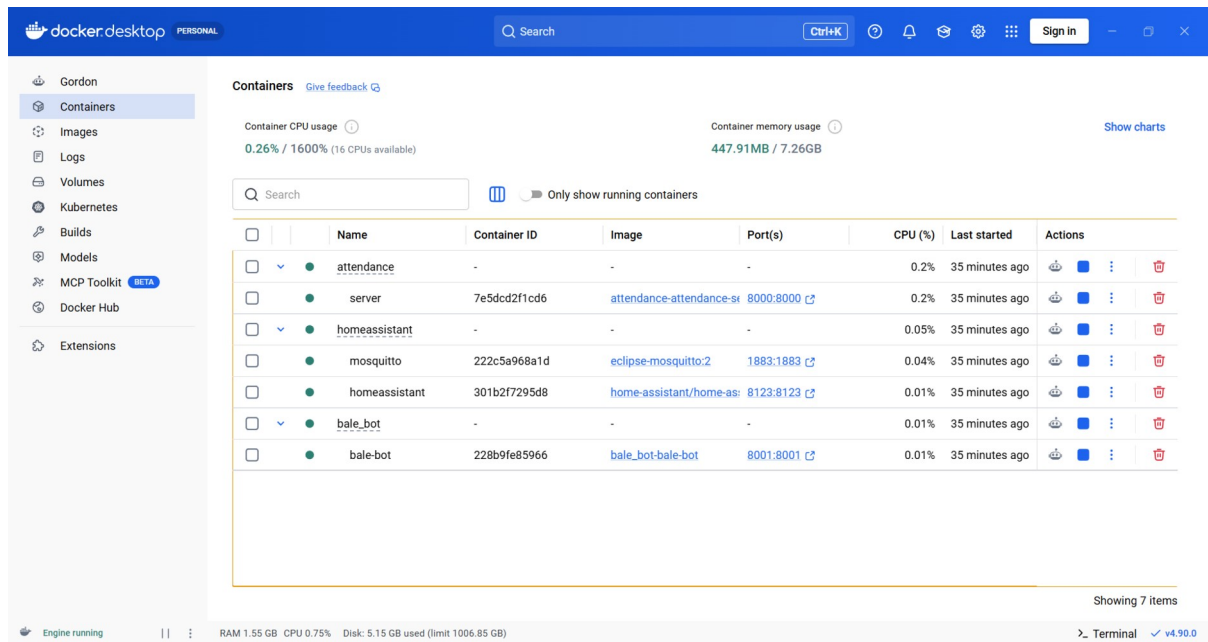


Figure 11-6: The system's running containers in Docker Desktop

Figure 11-7 shows the attendance archive in Google Sheets; each record is stored with a Jalali date, the exact time, the person's name and the entry/exit type, while a Bale notification is dispatched at the same moment. This archive is the optional delivery tier: during an Internet outage the records persist in the durable on-board queue and are sent in order once connectivity returns.

The screenshot shows a Google Sheets interface with a spreadsheet titled "Smart Home Attendance". The spreadsheet contains a table named "Table1" with the following data:

	تاریخ	Tr	ساعت	Tr	نام	نوع
2	1405/07/12		11:24:51		Soroush Rabiei	ورود
3	1405/07/12		11:25:43		Soroush Rabiei	خروج
4	1405/07/12		11:40:19		Soroush Rabiei	ورود
5	1405/07/12		21:15:46		Soroush Rabiei	ورود
6	1405/07/12		21:15:54		Soroush Rabiei	خروج
7	1405/07/14		11:55:08		Soroush Rabiei	ورود
8	1405/07/14		11:55:08		Soroush Rabiei	ورود
9	1405/07/14		11:55:13		Soroush Rabiei	خروج

Figure 11-7: The attendance archive in Google Sheets with Jalali dates

Chapter 12: Summary and Future Work

12.1 Summary

In this project a complete IoT-based smart-home system was designed and implemented whose distinguishing feature is bringing artificial intelligence to a constrained edge device and reusing the user's existing hardware for authentication: face recognition runs entirely on the microcontroller, the camera and the rich interface are borrowed from the user's smartphone, and the infrastructure (MQTT and Home Assistant) is deployed fully locally with Docker. The completed components are: the ESP32-S3 node with its touch display and secure web server; the on-chip face pipeline with multi-sample enrollment; auto-discovery integration with 13 HA entities; the behavioral learning agent with offline pre-training and online learning (94.2% and 93.6% accuracy); the three-tier attendance system with a durable queue, a Jalali sheet and Bale notifications; and the reliability mechanisms (watchdogs and network-aware reconnection).

From a learning perspective, the project was deliberately designed as a practical rehearsal of most specialized undergraduate courses: logic design and electronics (buses, relay, pull-up resistors), computer architecture and hardware constraints (memory, PSRAM, partitioning), operating systems and multi-core programming (FreeRTOS, concurrency, watchdogs), networking and protocols (MQTT, HTTPS/TLS, DNS and DHCP), databases and software engineering (modular architecture, a durable queue, Git and branch-based development), artificial intelligence (model quantization, TinyML, online learning), and interface and user-experience design. The final product, beyond the system itself, is a stable laboratory platform for developing and testing future ideas — from image recognition and voice commands to behavioral analysis of sensor data.

12.2 Future Work

The following development paths are proposed for the project:

1. Federated learning on microcontrollers: applying federated learning to train behavioral models jointly across several boards in a home, without raw data leaving the devices — the natural continuation of this project's privacy-preserving intelligence-at-the-edge approach.
2. Voice intelligence: adding on-chip voice-command recognition with the ESP-SR framework as a fifth interaction channel.

3. Sensor behavioral analytics: extending the learning agent to real presence (PIR HC-SR501) and light-intensity (BH1750) sensors and a real two-channel relay fan.
4. Fingerprint as a second authentication factor: adding a fingerprint sensor for phone-free entry and two-factor authentication.
5. Hardening the MQTT broker (authentication and TLS) and upgrading web authentication to the WebAuthn standard on top of the existing tokens.
6. Liveness detection to harden face recognition against photographs and video replay.
7. Wireless firmware updates (OTA) for easier maintenance.
8. An LVGL simulator on the computer (WSL2 with SDL2) for a faster UI development cycle.
9. Improving the Home Assistant dashboard (charts, automations and face management from HA) and a model-retrain button.
10. Final deployment on permanent hardware: a real electric lock, an enclosure with clean wiring, I2C pull-up resistors, and moving the services to a low-power always-on computer.
11. Complementary interaction options: BLE/iBeacon and NFC for close-range entry, and a dedicated mobile app.
12. Designing a custom printed circuit board (PCB) for the product version of the system.

References

- [1] Espressif Systems, “ESP-IDF Programming Guide (v6.x).” [Online]. Available: <https://docs.espressif.com/projects/esp-idf/>
- [2] Espressif Systems, “ESP-DL: Deep Learning Library for ESP Series.” [Online]. Available: <https://github.com/espressif/esp-dl>
- [3] Espressif Systems, “human_face_detect / human_face_recognition Components Registry.” [Online]. Available: <https://components.espressif.com/>
- [4] FreeRTOS Team, “The FreeRTOS Reference Manual.” [Online]. Available: <https://www.freertos.org/>
- [5] LVGL, “Light and Versatile Graphics Library, v9.5 Documentation.” [Online]. Available: <https://docs.lvgl.io/>
- [6] OASIS Standard, “MQTT Version 3.1.1 Specification.” [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/>
- [7] Eclipse Mosquitto, “Mosquitto Documentation.” [Online]. Available: <https://mosquitto.org/documentation/>
- [8] Home Assistant, “MQTT Discovery Documentation.” [Online]. Available: <https://www.home-assistant.io/integrations/mqtt/>
- [9] Docker Inc., “Docker Documentation.” [Online]. Available: <https://docs.docker.com/>
- [10] E. Rescorla and T. Dierks, “The Transport Layer Security (TLS) Protocol Version 1.2,” RFC 5246, 2008.
- [11] FastAPI, “FastAPI Documentation.” [Online]. Available: <https://fastapi.tiangolo.com/>
- [12] SQLite Consortium, “SQLite Documentation.” [Online]. Available: <https://www.sqlite.org/docs.html>
- [13] P. Warden and D. Situnayake, “TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers,” O'Reilly Media, 2020.
- [14] B. McMahan et al., “Communication-Efficient Learning of Deep Networks from Decentralized Data (Federated Learning),” in Proc. AISTATS, 2017.
- [15] W3C / FIDO Alliance, “Web Authentication (WebAuthn): An API for accessing Public Key Credentials.” [Online]. Available: <https://www.w3.org/TR/webauthn/>
- [16] Bosch Sensortec, “BME280 Combined Humidity and Pressure Sensor Datasheet.” [Online]. Available: <https://www.bosch-sensortec.com/>
- [17] Goodix, “GT911 Capacitive Touch Panel Controller Datasheet.” [Online]. Available: <https://www.goodix.com/>
- [18] Sitronix Technology, “ST7796 320x480 TFT Controller Datasheet.” [Online]. Available: <https://www.sitronix.com.tw/>